

The Age of Agentic AI

The software paradigm shifted. Yours should too.

Matej Lukášik · April 2026

Hello, my name is Matej Lukášik, I'm a friend of Amity Age and today I would like to tell you a bit about Agentic AI.

The presentation is going to take about 20 to 30 minutes and will be more theoretical and during the rest of the hour we can do Q&A and get into the practical details.

Part I

The Big Picture

Let's have a look at the bigger picture first so that your reasoning will be grounded and you can make decisions with confidence.

We Live in an Information Age

which means that every company is a software company

No matter the industry you're in, a lot of the value you deliver and capture is determined by how effectively you use software.

The Evolution of Software Distribution

Era	Medium	Model
1980s - 2000s	  Floppy disks, CDs, DVDs	Physical distribution
2000s - 2025	 Web & SaaS	Cloud subscriptions
2025+	 Agents	On-demand generation

At the beginning of the era of personal computers, developing and distributing software was difficult and expensive.

First, we had to write it in clumsy programming languages and then physically distribute it on floppy disks, CDs or later DVDs.

Then came the SaaS revolution, when we could distribute software via the browser. Some of you might not be old enough to remember this, but the fact that companies did not have to buy physical copies of a set version of software but instead could sign up online, pay comparatively very low monthly fees and receive free updates of the software all the time was revolutionary.

Today, we are entering a new software paradigm, a paradigm of Agentic AI. In this era, only the most powerful and complicated applications are built by big companies and subscribed to using the SaaS model. The rest is built by on-demand. On a long enough time horizon, maybe even the browser will dissapear and all you do is talk to your agent.

Why is that?

The Cost of Software is Collapsing

Instead of:

Research tools → Compare pricing → Sign contract → Learn UI

You can:

```
$ claude "build me a tool that does X"  
# → Working software in minutes
```

- You used to:
- identify a problem in the company
 - research tools that would help you solve it
 - judge whether they are compatible with what you are already using
 - decide on the tier and the number of seats
 - onboard everyone in the company to use them.

AND STILL HOWEVER MUCH you tried, the software never did everything you needed, and you always thought "if only it could do X"

With the advent of capable coding agents like claude code or codex you can sit down and write the exact software you need in minutes.

KEY TAKEAWAY

change your thinking from:

"What tool should I subscribe to next to solve my problem?"

to:

"How do I specify this well enough so that my coding agent can build it in one shot?"

Here is the key takeaway: Change your thinking FROM, TO. Use few solid tools, like Google Docs, Trello, Slack and drive the information flows in between them using agents. Since agents can now use any tool via either CLI tools, MCP server or the browser itseld, it makes more sense to build your own integrations that exactly suit your needs rather than wait for companies to build them.

If there is a little online tool you would like to use, but you need to sign up for it (because there is an upsell or a premium feature funnel down the road), don't do it. Ask your agent to write the tool for you. It will cost you a few cents, but it will be exactly as you need it and it will be free forever.

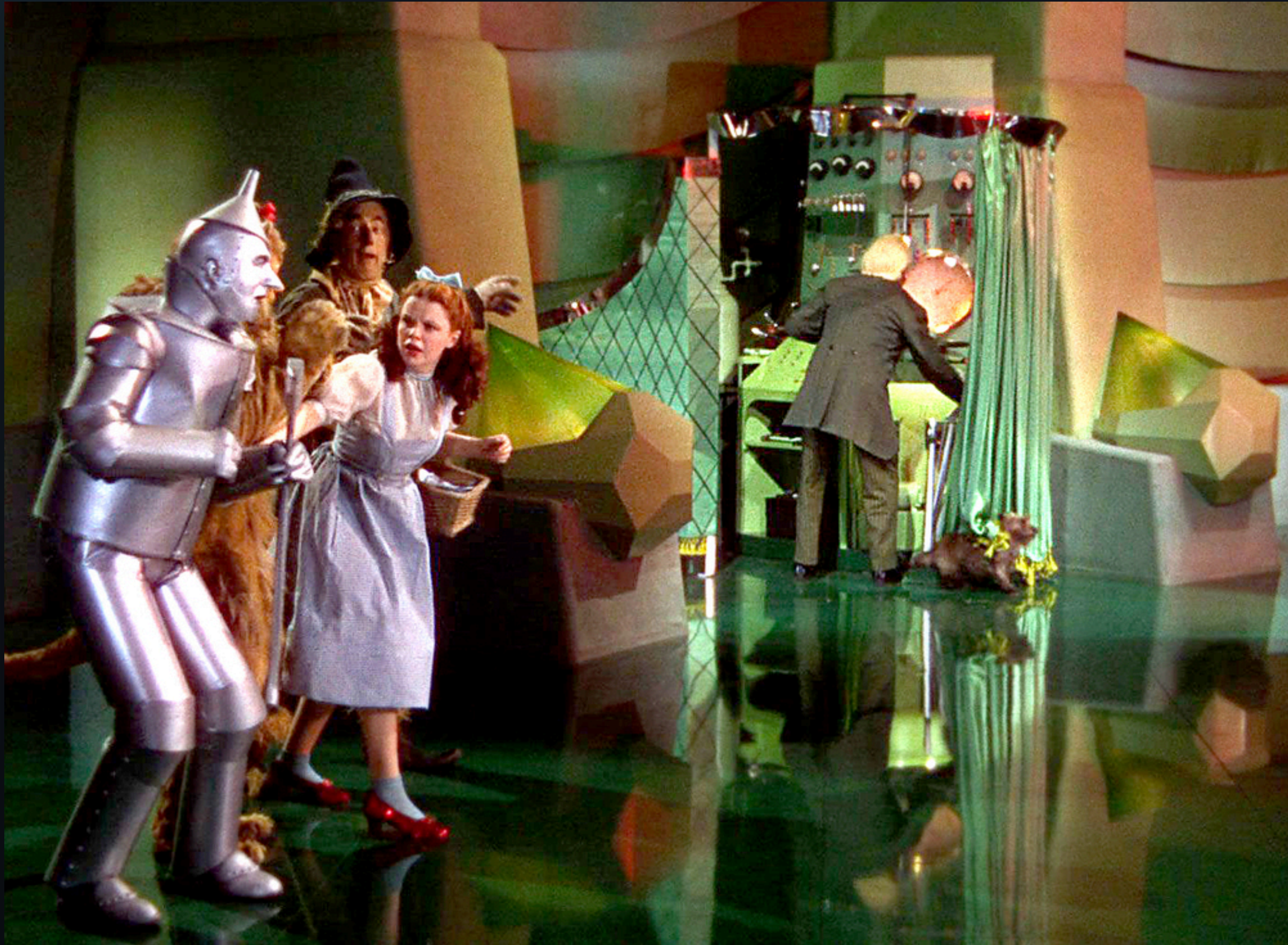
This is a subtle shift, Andrej Karpathy describes it as software becoming ephemeral, and it might take some time to click.

Part II

What is an AGENT?

One of the reasons we are talking today is that there is so much hype and almost a sense of FOMO around Agentic AI.

There is also the counternarrative of AI being a useless producing slop machine and stories of agentic workflows gone terribly wrong.



The truth is somewhere in the middle. Let me pull the curtain for you, walk you through what an agent exactly is.

Brain + Tools + Memory + Agency =
Agent

ChatGPT by itself is just an LLM chatbot, however, give it a brain, tools, memory and agency and it becomes a very capable Agent that can do a lot of knowledge work for you.

BRAIN -> writes plans, reviews them, reasons about the next best step

```

# Step 1: ask for a plan
planning_response = client.messages.create(
    model="claude-opus-4-6",
    system="You are an operations agent. When given a task, first output a numbered plan. Be specific.",
    messages=[{"role": "user", "content": "Assess whether our mining is profitable right now."}]
)

# model returns:
# "1. Fetch current BTC price
# 2. Fetch current network difficulty
# 3. Fetch our hashrate and power cost
# 4. Calculate revenue vs cost
# 5. Return a verdict"

plan_steps = parse_plan(planning_response) # split on numbered lines

# Step 2: execute each step, feeding results back as context
for step in plan_steps:
    response = client.messages.create(
        model="claude-opus-4-6",
        system="Execute this single step. Use tools if needed. Be concise.",
        messages=conversation + [{"role": "user", "content": f"Current step: {step}"}]
    )
    conversation.append({"role": "assistant", "content": response.content[0].text})

```

The intention here is not to go into the details and explain the code, but rather show you, that what we call an intelligent agent is a clever combination of good old structured workflows and LLMs.

TOOLS -> lets it browse the web, call APIs, write files, send messages, execute code, etc.

```

# What YOU write (clean API call):
response = client.messages.create(
    model="claude-opus-4-6",
    tools=tools,
    messages=[{"role": "user", "content": "Is mining profitable right now?"}]
)

# What the model ACTUALLY sees (roughly – Anthropic injects this into the context):
"""
You have access to the following tools:

<tools>
  <tool>
    <name>get_btc_price</name>
    <description>Fetch the current Bitcoin price in USD</description>
    <input_schema>{"type": "object", "properties": {}}</input_schema>
  </tool>
  <tool>
    <name>get_hashrate</name>
    <description>Fetch current hashrate from the mining dashboard</description>
    <input_schema>{"type": "object", "properties": {"rig_id": {"type": "string"}}}</input_schema>
  </tool>
</tools>

If a tool is appropriate, respond with a tool_use block.
If no tool is needed, respond directly.

User: Is mining profitable right now?
"""

# Model responds with:
# {"type": "tool_use", "name": "get_btc_price", "input": {}}
# – your code executes it, result goes back into context, loop continues

```

MEMORY -> keeps context across time

Normally an LLM only remembers what is in it's current context and we have to reset it when we reach the context window. We can create note files and then later load them into the context. However, the real nice way to do this, is to periodically save summarizations of our conversations in a vector database and then search this database on the fly thus seemingly remembering all we ever talked about.

```

import anthropic
import numpy as np

client = anthropic.Anthropic()

# --- WRITE: storing a memory ---
fact = "Rig #12 was replaced on 2025-11-03 due to a failed PSU."

embed_response = client.messages.create( # in practice: use a dedicated embeddings API
    model="claude-opus-4-6",
    # e.g. OpenAI embeddings or a local model
    ...
)
vector = get_embedding(fact) # → [0.021, -0.843, 0.374, ...] (1536 floats)
vector_db.store(vector, metadata={"text": fact, "date": "2025-11-03", "rig": 12})

# --- READ: at the start of every agent run ---
query = "Is rig #12 behaving normally?"
query_vector = get_embedding(query)

# fetch the top 3 most semantically similar memories
results = vector_db.search(query_vector, top_k=3)
# → ["Rig #12 was replaced on 2025-11-03 due to a failed PSU.",
#     "Rig #12 average hashrate: 98 TH/s",
#     "Rig #12 flagged for elevated temp on 2025-10-28"]

# inject only the relevant memories into the system prompt
system_prompt = f"""You are an operations agent.

Relevant context from memory:
{chr(10).join(results)}

Use this context when answering. Do not hallucinate hardware history."""

```

AGENCY -> means it can decide and act in a loop

```

messages = [{"role": "user", "content": user_input}]

while True:
    response = client.messages.create(
        model="claude-opus-4-6",
        system=system_prompt,
        tools=tools,
        messages=messages
    )

    if response.stop_reason == "end_turn":
        print(response.content[0].text)
        break
        # agent is done

    if response.stop_reason == "tool_use":
        tool_call = response.content[0]
        result = execute_tool(tool_call.name, tool_call.input) # your code runs here

        messages.append({"role": "assistant", "content": response.content})
        messages.append({"role": "user", "content": [{
            "type": "tool_result",
            "tool_use_id": tool_call.id,
            "content": str(result)
        }]])
        # ↑ loop continues – model sees the result and decides next step

```

KEY TAKEAWAY

LLMs are not agents

Agents use LLMs to achieve your goals

Here is a metaphor to help you think about this: You own your computer, you can take it anywhere, you can upgrade it's HW, install anything on it. However, you always need electricity to power it. In the same way, you should think about your agents, they should be yours, on your file systems and ideally run on your HW. To power them, instead of electricity, you supply them with intelligence. Intelligence is provided by different companies (openai, anthropic), that are competing for your dollars. You can choose different LLMs for different agents and different tasks based on their capabilities and pricing.

From a perspective of a Bitcoin company like Amity Age, this paradigm also allows you to spend satoshis for intelligence and get a fair discount for doing so by using solutions like routstr.

However, this is the exact same dilemma as with Bitcoin custody, the more sovereign you want the setup to be, the more work and inconvenience for you. On the other hand there are whole AI suites like the Claude Cowork that provide high levels of convenience but lock you into a vendor and can become more expensive.

As you start using agents and discover their power the costs will add up. Thinking about costs before implementing agentic workflows in your company is a valid concern.

Overall, this is a decision you will need to make: What levels of sovereignty and self-sustainability vs convenience do we want and what are we willing to pay for it?

Part III

The Use Cases

OK, now that we understand there is nothing magical about agents and AGI is not here yet, let's look at the most common ways agents are used in companies. As we move through them, I will also point out different patterns and paradigm shifts that I can see emerging.

Inbox Triage + Drafting

- Give your agent access to your inbox, it can classify emails, extract fields, forward if necessary and draft responses so that you can later just check them and send them
- If there is anything unusual or he is not sure, he can escalate to you via another channel
- Slowly over time you can teach the agent to treat your inbox just like you would
- Amity Age example: You get inbound emails from potential mining clients asking about hosting prices, availability in Norway vs Ethiopia, setup timelines etc. An agent could classify these by intent (new client inquiry, existing client support, partnership proposal), extract key details (desired hashrate, location preference, budget), draft a personalized response using your standard pricing. You can then review the email and send it yourself.

Failure modes: Wrong classification, wrong tone, too much access, accidental external reply.

Calendar Scheduling + Coordination

- You give agent access to your calendar and then ask it to schedule meetings using your voice via Telegram or any other channel. This is how we all imagined Siri would work.
- It checks for conflicts, sends invites, follows up if unanswered etc.
- Amity Age example: Your company operates across many timezones. An agent can handle all the scheduling across these regions, respect local holidays and working hours, etc.

Failure modes: Auth friction, time-zone errors, missing buffers, double-booking.

Research

- Research agent gathers sources, summarizes findings, and turns them into briefs for you to read any time.
- When preparing the list of use cases we are going through right now, I asked my agent to search for the 30 most popular YT videos on agent use cases, transcribe them all and then extract the most common use cases.

Failure modes: Hallucinations, weak sources, missing citations, drifting away from the rubric.

Monitoring + Alerts

- For things that have clear definitions and metrics, like for example bitcoin price, it is easy to set an alert.
- But for things that are harder to define and judgment is needed, like monitoring news or competition etc. monitoring and alerting is more difficult.

Failure modes: Brittle scraping, noisy alerts, and Terms-of-Service constraints.

1.

▲

LittleSnitch for Linux (obdev.at)

1071 points by pluc 14 hours ago | hide | 363 comments
2.

▲

Clean code in the age of coding agents (yanist.com)

22 points by yanis_t 46 minutes ago | hide | 20 comments
3.

▲

A WebGPU Implementation of Augmented Vertex Block Descent (github.com/jure)

44 points by juretriglav 3 hours ago | hide | 2 comments
4.

▲

Help Keep Thunderbird Alive (thunderbird.net)

315 points by playfultones 7 hours ago | hide | 211 comments
5.

▲

Meta removes ads for social media addiction litigation (axios.com)

145 points by giuliomagnifico 1 hour ago | hide | 67 comments
6.

▲

Session is shutting down in 90 days (getsession.org)

46 points by balamatom 2 hours ago | hide | 68 comments
7.

▲

Building a framework-agnostic Ruby gem (and making sure it doesn't break) (masilotti.com)

13 points by joemasilotti 1 hour ago | hide | 1 comment
8.

▲

Wit, unker, Git: The lost medieval pronouns of English intimacy (bbc.com)

93 points by eigenspace 5 hours ago | hide | 53 comments
9.

▲

How Pizza Tycoon simulated traffic on a 25 MHz CPU (pizzalegacy.nl)

119 points by FinnKuhn 2 hours ago | hide | 26 comments
10.

▲

Introduction to Nintendo DS Programming (patater.com)

78 points by medbar 5 hours ago | hide | 12 comments
11.

▲

FreeBSD Laptop Compatibility: Top Laptops to Use with FreeBSD (frebsdoundation.github.io)

72 points by fork-bomber 6 hours ago | hide | 43 comments
12.

▲

Open Source Security at Astral (astral.sh)

288 points by vinfnx 11 hours ago | hide | 66 comments
13.

▲

Lichess and Take Take Take Sign Cooperation Agreement (lichess.org)

59 points by stevage 3 hours ago | hide | 6 comments
14.

▲

Show HN: CSS Studio. Design by hand, code by agent (cssstudio.ai)

67 points by SirHound 3 hours ago | hide | 59 comments
15.

▲

Show HN: 41 years sea surface temperature anomalies (willhelps.org)

109 points by willmeyers 2 hours ago | hide | 36 comments
16.

▲

Haunted Paper Toys (ravensblight.com)

177 points by exvi 10 hours ago | hide | 23 comments
17.

▲

Launch HN: Relvy (YC F24) – On-call runbooks, automated (relvy.ai)

17 points by behat 3 hours ago | hide | 13 comments
18.

▲

Tree Calculus (treecalcul.us)

53 points by tosh 5 hours ago | hide | 14 comments
19.

▲

Claude mixes up who said what (dwyer.co.za)

270 points by sixhobbits 5 hours ago | hide | 252 comments
20.

▲

Small Engines (scottlocklin.wordpress.com)

24 points by surprisetalk 2 hours ago | hide | 6 comments
21.

▲

Show HN: Moon simulator game, ray-casting (mooncraft2000.com)

61 points by JKCalhoun 6 hours ago | hide | 13 comments
22.

▲

Creating the Futurescape for the Fifth Element (2019) (theasc.com)

82 points by nixass 6 hours ago | hide | 47 comments
23.

▲

Dr. Dobb's Developer Library DVD 6 (archive.org)

103 points by kristianp 10 hours ago | hide | 40 comments
24.

▲

USB for Software Developers: An introduction to writing userspace USB drivers (werwolv.net)

367 points by WerWolv 19 hours ago | hide | 40 comments
25.

▲

I ported Mac OS X to the Nintendo Wii (bryankeller.github.io)

1753 points by blkhp19 23 hours ago | hide | 296 comments
26.

▲

Understanding the Kalman filter with a simple radar example (kalmanfilter.net)

393 points by alex_be 22 hours ago | hide | 54 comments

Here is an example. Things in AI are moving so fast, you have to check the news every day. But I don't want to spend my time going through the list of news and spend my cognitive cycles on whether this particular news piece is relevant to AI or not. I asked my agent to go to this website every day, and extract only the news that are relevant to AI and send them to me via Telegram.

Pull, Don't Poll

Agents can

- watch inboxes, dashboards, feeds, and databases
- pull out what changed
- summarize what matters
- route it to the right person
- take the next step when allowed

The interface matters less. The outcome matters more.

This way of monitoring data feeds is another subtle paradigm shift. The browser makes humans go check feeds for updates. With agents, the software starts working in the background and brings you the update so that you don't have to waste your time on evaluating whether it's relevant for you or not.

Knowledge Base

- You connect your company documents, standard operating procedures, contracts and internal knowledge and let the agent answer repetitive customer questions
- It can either answer directly or draft a response for a human to approve, depending on the risk level
- Amity Age example: Your Bitcoin Education Academy has accumulated a lot of teaching material — how Bitcoin works, how to avoid scams, how to handle objections. An agent trained on this knowledge base could answer student questions 24/7, help new educators prepare for sessions, or even power a Telegram bot for the Roatan education center that answers visitors' questions in English and Spanish.

Failure modes: Prompt injection, stale docs, brand-tone mistakes, compliance and logging gaps.

Lead Generation + Enrichment

- The agent finds leads, enriches company data, drafts personalized outreach and keeps the pipeline organized for you
- This is easy to measure and A/B test
- You can think of it as outsourcing the top of your sales funnel to software

Failure modes: Legal and ToS risk, low-quality data, spammy outreach, poor deliverability.

Document Processing

- You feed the agent receipts, invoices, contracts or scanned PDFs and it extracts the relevant fields into structured data

Failure modes: OCR edge cases, vendor template variance, and sensitive financial data handling.

Browser Automation

- Sometimes there is no API, and the only way to automate a workflow is to let the agent operate the browser directly
- It clicks through portals, fills forms, downloads files and completes repetitive web tasks that humans would otherwise do manually
- This unlocks the last mile of automation for many legacy systems

Failure modes: Brittle selectors, MFA friction, session expiry, costly mis-clicks.

Part IV

Tips

Alright, so I kind flew through that, because the ways you are going to use agents are highly specific to you. Some of the use cases might be still difficult for you to imagine and that's OK.

But here are some tips that you can start using from day one.

Start turning everything into data

To fully leverage agents, turn as much as you can into text.

This way your agents can see it, reason about it, find connections and insight you might miss.

To fully leverage agents, turn as much as you can into text.

This way your agents can see it, reason about it, find connections and insights you might miss.

In your case, I would start leveraging Google Docs. Imagine this: Every day, most of the work you have done, is saved and made available in a Google Drive folder your agent can access. Every night, each of your agents scan through the work you have done and look for connections. If they see connections in between your work, they will let you know and make sure everyone is informed.

<!-- ![Presentation as an md file](presentation-as-an-md-file.png) -->

Human-in-the-Loop, Always

Never let agents act alone on high-stakes decisions

Agent drafts → Human reviews → Human approves →
Agent executes

This is the most important safety tip. Agents are powerful but they hallucinate, misunderstand context, and lack judgment. The pattern that works is: let the agent do the heavy lifting (research, drafting, data processing), but keep a human in the loop for anything that touches customers, money, or reputation. Start with agents that draft and you approve. Over time, as you build trust, you can expand their autonomy — but never fully remove the human check on things that matter. Think of it like hiring a very fast junior employee: you wouldn't let them send contracts to clients without review.

Invest in Prompts, Not Just Tools

Your agent is only as good as its instructions

The quality of the output is directly proportional to the quality of the prompt. Treat your best prompts like company assets. Version control them. Share them across the team. When you find a prompt that produces consistently good results — for drafting client emails, for generating reports, for analyzing data — save it and reuse it. This is the new form of institutional knowledge. A well-crafted prompt is worth a lot because it encodes exactly how your company thinks and operates.

Measure Before You Scale

Automate one workflow end-to-end before expanding

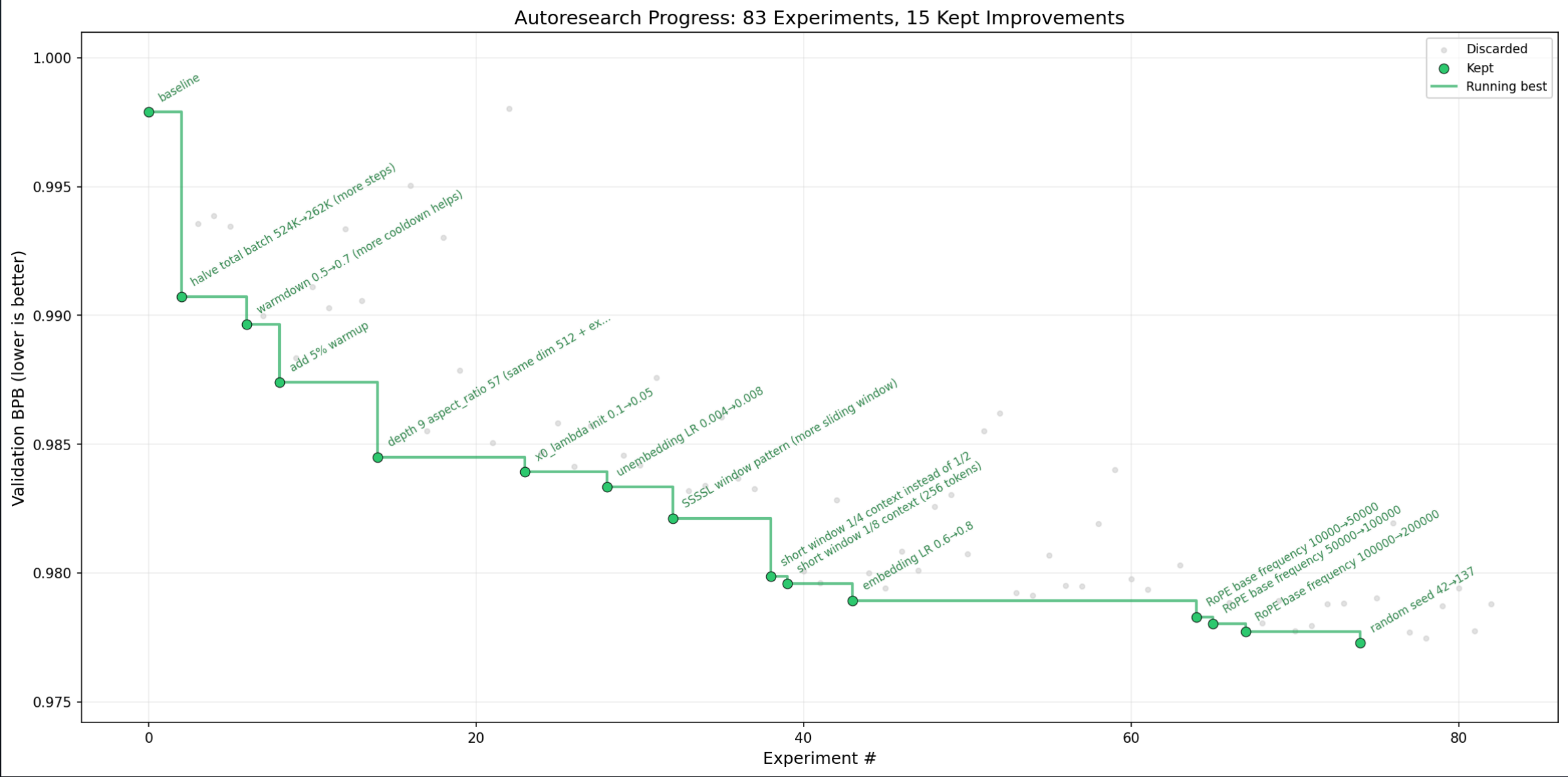
It is tempting to try to automate everything at once. Resist this. Pick one workflow — the most painful, repetitive one — and automate it properly. Measure the time saved, the error rate, the cost. Only then move to the next one. This gives you real data to make decisions with, builds confidence in the team, and prevents the chaos of half-built automations everywhere. A company with one fully automated workflow is in a much better position than one with ten half-working ones.

Autoresearch

This is the future: AI agents that improve themselves.

github.com/karpathy/autoresearch

And here is the best and most fascinating tip of all. See if you could use the autoresearch project by Andrej Karpathy. A tool that allows agents to improve themselves.



Look at this chart. On the X axis we have the number of experiments, on the Y axis we have the metric we are trying to improve. The substance of the metric is not important, what is important is that lower the metric the better. Andrej asked the agent to adjust the code he supplied, run the experiment for 5 minutes and see if the metric improved. If yes, the agent would keep the adjustment, if not, the agent would discard it. In either case it would try again and again and again. Overnight, it ran 83 Experiments, 15 of which were successful in lowering the metric.

How could Amity Age use this? I'm sure you have sales funnels. Let's say you have 1000 visitors to your website every day. Let the agent run an experiment on a daily basis. Change the copy of the website, the buttons, etc. and see if the conversion rate improves. If yes, keep the new copy, if not, change it. You can do the same for ads, or anything else. THE IMPORTANT THING HERE, is that there is ONE clear objective metric for the agent to improve and a set of variables he can change.

This is self-improving AI and the ones who master it are going to be at great advantage.

Part V

Actions You Can Take Now

OK, so that would be the theoretical side. Here are the super practical steps you can take tomorrow.

Start Tomorrow

1. Spin up a VPS with Docker

One container per employee with OpenClaw

2. Give each employee a token budget

\$100 worth of tokens is enough to explore and build intuition

3. Start small, find the friction

Ask each employee to find friction in their workflows. Look for repetitive tasks: reports, data extraction, formatting, scheduling, etc.

Part VI

How I Can Help?

Overall, I tried to give you a good foundation, so that you can be confident in taking the next steps by yourselves. However, if you feel like you need more help, here are some of the services I offer you.

Services

Consulting & Advisory

Assess workflows, identify high-impact automation opportunities, build adoption roadmap.

Implementation

Design and deploy custom AI agents tailored to your processes. From single-task automations to multi-agent orchestration.

Training & Workshops

Hands-on sessions for your team. From prompt engineering to production agent workflows. Build internal capability, not dependency.

The Bottom Line

The age of agentic AI is here.

But just like with everything else, there is a learning curve.

Don't FOMO, learn to walk before you can run.

Thank You

Matej Lukášik

Questions?